

B28: SQL SuperHero Program : Session 11

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

2 March 2025 at 05:40

ORDER BY

The ORDER BY clause in SQL is a powerful feature used to sort query results in either ascending or descending order based on one or more columns. ORDER BY has no impact on total number of records in table, it will help you just to arrange the data.

```
select * from hr.countries order by REGION_ID
```

COUNTRY_ID	COUNTRY_NAME	REGION_ID
NL	Netherlands	1
FR	France	1
UK	United Kingdom	1
DK	Denmark	1
BE	Belgium	1
CH	Switzerland	1
IT	Italy	1
DE	Germany	1
US	United States of America	2
CA	Canada	2

Data sorted
1 then 2
then 3
then 4

```
select * from hr.employees order by first_name
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL
121	Adam	Frapp	AFRIPP
196	Alana	Walsh	AWALSH
147	Alberto	Errazuriz	AERRAZUR
103	Alexander	Hunold	AHUNOLD
115	Alexander	Khoo	AKHOO
185	Alexis	Bull	ABULL

Data is sorted
on first name
column
A then B then C
:

```
select * from hr.countries order by region_id desc;
```

desc means display data in descending order

```
select * from hr.countries order by REGION_ID desc
```

COUNTRY_ID	COUNTRY_NAME	REGION_ID
ZM	Zambia	4
ZW	Zimbabwe	4
NG	Nigeria	4
KW	Kuwait	4
IL	Israel	4
EG	Egypt	4
CN	China	3
AU	Australia	3
SG	Singapore	3
MI	Malaysia	3

DESC
means
Highest to
lowest

order by can be used against any column on which we want to display.

=====

Why order By not worked with UNION

```
select * from test1 order by id
UNION
select * from test2 order by id;
```

```
select * from test1 order by id union
select * from test2 order by id
```

ORA-00933: SQL command not properly ended

Reason: Order BY is always the last operation. We are here doing UNION and UNION clause is basically take data from both data sets and then sort them and display the unique values. so order by need to run first, which is not possible. so, we will do change such type of query and do the ORDER by in last as below:

```
select * from (
select * from test1
UNION
select * from test2
) order by id;
```

```
select * from (
select * from test1
union
select * from test2
) order by id
```

↳ Last operation

ID
1
2
3
4
5
6

6 rows selected.

We can also use column number position while doing order by.

so if we do order by 3 desc, it means sorting the data by 3rd column in descending order. the 3rd position column is region_id, so it sort on that column.

```
select * from hr.countries order by 2
```

—→ 2 means
2nd column

① COUNTRY_ID	② COUNTRY_NAME	③ REGION_ID
AR	Argentina	2
AU	Australia	3
BE	Belgium	1
BR	Brazil	2
CA	Canada	2
CN	China	3
DK	Denmark	1
EG	Egypt	4
FR	France	1
DE	Germany	1

Sorted

```
select * from hr.countries order by 2 desc
```

① COUNTRY_ID	② COUNTRY_NAME	③ REGION_ID
ZW	Zimbabwe	4
ZM	Zambia	4
US	United States of America	2
UK	United Kingdom	1
CH	Switzerland	1
SG	Singapore	3
NG	Nigeria	4
NL	Netherlands	1

2nd column from output
&
sort by
descending order.

Null Handling with Order BY

If the null ordering is not specified then the handling of the null values is:

- **NULLS LAST** if the sort is **ASC**
- **NULLS FIRST** if the sort is **DESC**

If neither ascending nor descending order is specified, and the null ordering is also not specified, then both defaults are used and thus the order will be ascending with NULLS LAST.

--> By default NULLS records comes in Last when we do sort by ASC

```
select * from countries order by 3
```

order by 3 → by default asc
order

ZM	Zambia	4
NG	Nigeria	4
KW	Kuwait	4
AA	ABCD	-

→ null in
last

26 rows selected.

```
select * from countries order by 3 desc
```

COUNTRY_ID	COUNTRY_NAME	REGION_ID
AA	ABCD	-
EG	Egypt	4
ZM	Zambia	4
NG	Nigeria	4
KW	Kuwait	4
TI	Tanzania	4

Desc order
null first

```
select * from countries order by 3 nulls first
```

COUNTRY_ID	COUNTRY_NAME	REGION_ID
AA	ABCD	-
FR	France	1
BE	Belgium	1
DK	Denmark	1
CH	Switzerland	1
IT	Italy	1
DE	Germany	1

Now
it come
first

```
select * from countries order by 3 nulls first
```

1 COUNTRY_ID	2 COUNTRY_NAME	3 REGION_ID
AA	ABCD	-
FR	France	1
BE	Belgium	1
DK	Denmark	1
CH	Switzerland	1

coming
first

=====

1st do desc on 3rd column

```
select * from countries order by 3 desc, 2 asc
```

<i>1</i> COUNTRY_ID	<i>2</i> COUNTRY_NAME	<i>3</i> REGION_ID
AA	ABCD	-
EG	Egypt	4
IL	Israel	4
KW	Kuwait	4
NG	Nigeria	4
ZM	Zambia	4
ZW	Zimbabwe	4

on top of the output do asc on 2nd column

=====

```
create table test (name varchar2(10));
```

```
insert into test values ('A');
insert into test values ('B');
insert into test values ('C');
insert into test values ('a');
insert into test values ('b');
insert into test values ('c');
insert into test values ('D');
```

Can Order By Ignore Case? Like in the above query, if we do why upper case will come first and then lower case?

```
1 select * from test order by name;
2
3
4
```

NAME
A
B
C
D
a
b
c

} - Capital letter first

} - small letter after that ?

Why

Download CSV
7 rows selected.

Because, ASCII value of 'a' is more than 'A'. So, while sorting normal it took ascending order and choose to print 'A' first.

```
select ascii('A') cap_a, ascii('a') small_a from dual
```

CAP_A	SMALL_A
65	97

'a' has more value
than 'A'

WITH CLAUSE

--> Just introduced WITH CLAUSE a bit. Basically it is Subquery Factoring.

- The SQL WITH clause allows you to give a sub-query block a name, which can be referenced in several places within the main SQL query.
- WITH clause enables us to write our logic as a block and the data extracted in this block act as a temporary table and is available in the cache. So next time when the query is referenced, oracle doesn't need to execute it rather it directly refers the data extracted earlier. Thus, adds on to performance improvement. Always try to create a block of 'WITH' for the query, if the query result is to reference again and again.
- The same Having clause query, let's try to do via with clause

Syntax is :

WITH <name of CTE> AS (select query)

It is very helpful when you need the same set of results data multiple times. In such a case you can simply define a CTE for this data and reuse the same again and again by referencing it. It's a kind of code reuse.

--> Suppose I have to find out the highest salary of each department who has more than 10000 as highest salary

In this question, I divided my problem in 2 part:

- **Find out the highest salary of each department**
- **When first done then choose only those who have more than 10000**

```
select max(salary), department_id from hr.employees group by department_id)
```

This is the query , which will give me maximum salary for each department. Very straightforward sql, but lets give it a name. Lets call it Tabish. (He is a participant in B27 your batch. You can take any name);

```
with tabish as(  
  select max(salary), department_id from hr.employees group by department_id)  
select * from tabish
```

Now, the above will create a structure in memory called Tabish. Now we have to pick data from Tabish where salary > 10000

with CTE as(
 select max(salary) maxsal, department_id from hr.employees group by department_id)
 select * from CTE where maxsal > 10000

→ with clause. any name

→ I have added condition here.
 > 10000

MAXSAL	DEPARTMENT_ID
12008	110
24000	90
11000	30
13000	20
12008	100
14000	80

6 rows selected.

Details of Employee who has maximum salary

with n1 as (
 select max(salary) salary from hr.employees
) select * from n1

→ just find the maximum salary first

SALARY
24000

with n1 as (
 select max(salary) salary from hr.employees
) select E.*
 from n1, hr.employees E
 where n1.salary = E.salary

1st Part

2nd

just join with main table on salary (who has the 50)

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMI
100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000	-

Suppose, I have to find the employees from each department, who are taking the maximum salary. I want all details. I can not do all in selection, because then I have to do everything in group by as well.
 So, Lets find the max salary from each department and then I will join with main table to see who is taking that salary in that department.

combining
query

```
with AG as (
  select max(salary) maxsal, department_id from hr.employees group by department_id
) select E.employee_id, E.first_name, e.last_name, e.hire_date, ag.department_id
from AG, hr.employees E
where ag.department_id = e.department_id
and ag.maxsal = e.salary
```

1st part
↓
max(sal) from
each
department

join that result
with main
table

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	HIRE_DATE	DEPARTMENT_ID
100	Steven	King	17-JUN-03	90
103	Alexander	Hunold	03-JAN-06	60
108	Nancy	Greenberg	17-AUG-02	100
114	Den	Raphaely	07-DEC-02	30
121	Adam	Fripp	10-APR-05	50
145	John	Russell	01-OCT-04	80

Create table test (Name varchar2(20), class number, percentage number);

```
insert into test values ('Ravi',1,80);
insert into test values ('Ravi',2,85);
insert into test values ('Ravi',3,90);
insert into test values ('Ravi',4,95);
insert into test values ('Ravi',5,77);
insert into test values ('Kishan',1,80);
insert into test values ('Kishan',2,90);
```

```
select * from test1
```

NAME	CLASS	PERCENTAGE
Ravi	1	80
Ravi	2	85
Ravi	3	90
Ravi	4	95
Ravi	5	77
Kishan	1	80
Kishan	2	90

7 rows selected.

Write a SQL to find the percentage obtained by each student in a recent class?

Break query in 2 parts: The first part lets find out the recent class of each student. simple, each means group by. so for each name, we find the max class and the query returns me 2 rows.

```
with N1 As (  
  select max(class) class, Name from test1 group by Name  
) select * from N1
```

CLASS	NAME
2	Kishan
5	Ravi

2 rows selected.

Now when I got 2 rows , just join this with main table to get the percentage associated with this name and this class

```
with N1 As (  
  select max(class) class, Name from test1 group by Name  
) select N1.Name , test1.PERCENTAGE  
from N1, test1  
where N1.Name = test1.name  
and N1.class = test1.class
```

Join with main table

NAME	PERCENTAGE
Ravi	77
Kishan	90

2 rows selected.

Question Discussed In class

Product_In		Product_Out	
Productid	Qty	Productid	Qty
1	10	1	30
1	40	2	20
2	70	2	40
3	30	3	30
4	60		

Output			
I_PID	I_Qty	O_Qty	balance
1	50	30	20
2	70	60	10
3	30	30	0
4	60	0	60

If You see the above image, we have 2 tables product_In and Product_OUT and we need output as expected in image. Lets create the table first

create table product_in(Product_id number, qty number);

Insert Into product_in values (1,10);

Insert Into product_in values (1,40);

Insert Into product_in values (2,70);

Insert Into product_in values (3,30);

Insert Into product_in values (4,60);

create table product_out(Product_id number, qty number);

Insert Into product_out values (1,30);

Insert Into product_out values (2,20);

Insert Into product_out values (2,40);

Insert Into product_out values (3,30);

Understand this Output, what exactly they want:

Product_In		Product_Out	
Productid	Qty	Productid	Qty
1	10	1	30
1	40	2	20
2	70	2	40
3	30	3	30
4	60		

Output			
I_PID	I_Qty	O_Qty	balance
1	50	30	20
2	70	60	10
3	30	30	0
4	60	0	60

50-30

70-60

30-30

60-0

- If you try to understand , what exactly happening is : In 1st table product_IN , we have few records for each product and their qty and in output table, we add them together.
- in 1st table product_id =1 have 2 rows of 10,40 and in output it is showing 1 line product_id = 1 and qty = 50
- they just sum the qty for each product. Each means group By

```

1 select product_id, sum(qty) I_QTY from product_in group by product_id;
2
3
4

```

PRODUCT_ID	I_QTY
1	50
2	70
4	60
3	30

Download CSV
4 rows selected.

- check the output, half part we achieved. now stop here
- lets go to other part in other table product_out. it is happening same thing.
- see the O_QTY column, for 1 it is 30, but for 2 it is 20+40 = 60 and for 3 it is 30 and nothing for 4 so display 0
- lets make it

SQL Worksheet

```

1 select product_id, sum(qty) O_QTY from product_out group by product_id;
2
3
4

```

PRODUCT_ID	O_QTY
1	30
2	60
3	30

Download CSV
3 rows selected.

now we have both set done, let see how to combine them together. as we started with clause in last session. with clause help you to divide and solve approach we have 2 things consider both as 2 different table

`with vishal1 as (select product_id, sum(qty) I_QTY from product_In group by product_id) select * from vishal1`

same first query, I just added **With vishal1 as**

it automatically create a structure in memory names vishal1, actually thetre is nothing vishal1 in real, but just in memory it created to make our query writing life simple

same for 2nd part, now we have 2 different select, which are giving us 1st and 2nd part of data, now we have to combine them. with clause gives us flexibility that we can write 2-3-4-5-6 as many select we want and combine all together in 1 statement

```

with vishal1 as (select product_id, sum(qty) I_QTY from product_in
group by product_id),
vishal2 as (select product_id, sum(qty) O_QTY from product_out
group by product_id)

```

now we can see both data we have in a query, let see how we combine. actually consider vishal1 and vishal2 as a table and join them. I have not started JOIN yet, so do not worry who does not know joins

```

with vishal1 as (select product_id, sum(qty) I_QTY from product_in
group by product_id),
vishal2 as (select product_id, sum(qty) O_QTY from product_out
group by product_id) select vishal1., vishal2.
from Vishal1 left join vishal2 on vishal1.product_id = vishal2.product_id

```

```

1 with vishal1 as (select product_id, sum(qty) I_QTY from product_in
2 group by product_id),
3 vishal2 as (select product_id, sum(qty) O_QTY from product_out
4 group by product_id) select vishal1.*, vishal2.*
5 from Vishal1 left join vishal2 on vishal1.product_id = vishal2.product_id

```

PRODUCT_ID	I_QTY	PRODUCT_ID	O_QTY
1	50	1	30
2	70	2	60
3	30	3	30
4	60	-	-

Download CSV
4 rows selected.

IQty - OQty wants

we don't want this column in final output

Input:

Students table:

student_id	student_name
1	Alice
2	Bob
13	John
6	Alex

Subjects table:

subject_name
Math
Physics
Programming

Examinations table:

student_id	subject_name
1	Math
1	Physics
1	Programming
2	Programming
1	Physics
1	Math
13	Math
13	Programming
13	Physics
2	Math
1	Math

Write a solution to find the number of times each student attended each exam.

Return the result table ordered by student_id and subject_name.

Output:

student_id	student_name	subject_name	attended_exams
1	Alice	Math	3
1	Alice	Physics	2
1	Alice	Programming	1
2	Bob	Math	1
2	Bob	Physics	0
2	Bob	Programming	1
6	Alex	Math	0
6	Alex	Physics	0
6	Alex	Programming	0
13	John	Math	1
13	John	Physics	1
13	John	Programming	1

```

Create table student (
Student_id number,
Student_Name Varchar2(20)
);

```

```

Insert into student Values (1, 'Alice');
Insert into student Values (2, 'Bob');
Insert into student Values (13, 'John');
Insert into student Values (6, 'Alex');

```

```

Create table examination (
Student_id number,
Subject_Name Varchar2(20)
);

```

```

Insert into examination Values (1, 'Math');

```

```

Insert into examination Values (1, 'Physics');
Insert into examination Values (1, 'Programming');
Insert into examination Values (2, 'Programming');
Insert into examination Values (1, 'Physics');
Insert into examination Values (1, 'Math');
Insert into examination Values (13, 'Math');
Insert into examination Values (13, 'Programming');
Insert into examination Values (13, 'Physics');
Insert into examination Values (2, 'Math');
Insert into examination Values (1, 'Math');

```

```

Create table Subject (
Subject_Name Varchar2(20)
);

```

```

Insert into Subject Values ('Math');
Insert into Subject Values ('Physics');
Insert into Subject Values ('Programming');

```

we have 4 student, came with all 3 subject

Output:

student_id	student_name	subject_name
1	Alice	Math
1	Alice	Physics
1	Alice	Programming
2	Bob	Math
2	Bob	Physics
2	Bob	Programming
6	Alex	Math
6	Alex	Physics
6	Alex	Programming
13	John	Math
13	John	Physics
13	John	Programming

Cross join

⇒ table 1 & 2 Cross

```

select S.*, sub.subject_name
from student S, subject sub
order by 1,3; --- Order by we will learn in next class, just focus on cross join here

```

Student table have 4 records and subject have 3, so total $4 \times 3 = 12$ records
This will give you the first 3 columns of your output.

STUDENT_ID	STUDENT_NAME	SUBJECT_NAME
1	Alice	Math
1	Alice	Physics
1	Alice	Programming
2	Bob	Math
2	Bob	Physics
2	Bob	Programming
6	Alex	Math
6	Alex	Physics
6	Alex	Programming
13	John	Math
13	John	Physics
13	John	Programming

First 3 columns of output is here.

now, lets work for 4th column. How many times they give exam for each students. each means : group by. There should be no confusion ever. Each means Group by. If we see the table EXAMINATIONS, we have to count how many times student is coming with subject.

select STUDENT_ID,SUBJECT_NAME, count(*) attended_exams from examination group by STUDENT_ID,SUBJECT_NAME

```
select STUDENT_ID,SUBJECT_NAME, count(*) attended_exams from examination group by STUDENT_ID,SUBJECT_NAME
```

STUDENT_ID	SUBJECT_NAME	ATTENDED_EXAMS
2	Programming	1
2	Math	1
13	Math	1
1	Math	3
1	Physics	2
13	Programming	1
13	Physics	1
1	Programming	1

8 rows selected.

This column we want in final output

Now, if you see, we got 3 columns from 1st query and 1 column from this query and both columns have Student_id and Subject_name common between these. So, lets use our beloved WITH clause. and combine both queries

```
with n1 as (
  select S.*, sub.subject_name
  from student S, subject sub
  order by 1,3
),
n2 as (
  select STUDENT_ID,SUBJECT_NAME, count(*) attended_exams from examination group by STUDENT_ID,SUBJECT_NAME
) select n1.*, n2.attended_exams
from n1, n2
where n1.STUDENT_ID = n2.STUDENT_ID
and n1.SUBJECT_NAME = n2.SUBJECT_NAME;
```

STUDENT_ID	STUDENT_NAME	SUBJECT_NAME	ATTENDED_EXAMS
1	Alice	Math	3
1	Alice	Physics	2
1	Alice	Programming	1
2	Bob	Math	1
2	Bob	Programming	1
13	John	Math	1
13	John	Physics	1
13	John	Programming	1

8 rows selected.

we got it, but something missing

We got the data, but something is missing from Final output (Refer the question). Student id = 6 is not coming and student_id = 2 with subject physics is not coming. Why? We have total 12 records from N1 and only 8 records from n2 and we did an inner join. So, lets try left join, because we want all data from N1.

```
with n1 as (
  select S.*, sub.subject_name
  from student S, subject sub
  order by 1,3
),
n2 as (
  select STUDENT_ID,SUBJECT_NAME, count(*) attended_exams from examination group by STUDENT_ID,SUBJECT_NAME
```

```
) select n1.*, n2.attended_exams  
from n1 left join n2  
on( n1.STUDENT_ID = n2.STUDENT_ID  
and n1.SUBJECT_NAME = n2.SUBJECT_NAME)
```

```
with n1 as (  
  select S.*, Sub.subject_name from student S ,subject sub  
  order by 1,3  
) ,  
N2 As (select S.*, E.Subject_name, Count(*) attended_exam  
  from student S inner Join examination E on S.Student_id = E.Student_id  
  group by S.Student_id, S.STUDENT_NAME, E.SUBJECT_NAME)  
select N1.*, N2.attended_exam  
from N1 Left join N2 On (N1.Student_id = N2.Student_id and N1.Subject_name = N2.Subject_name)
```

Handwritten annotations:

- Green bracket next to the first query block:] - N1
- Green bracket next to the second query block:] - N2
- Red bracket under the join condition: All from N1 & matching n2